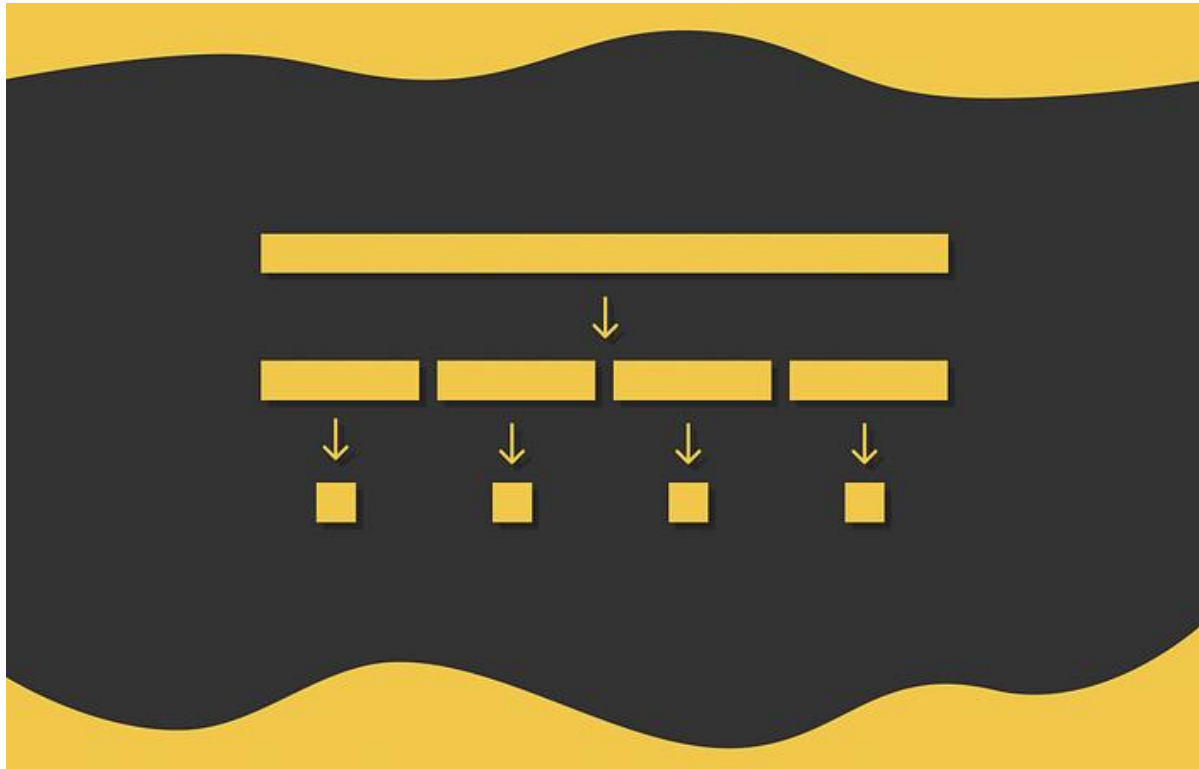


Similarity Search, Part 2: Product Quantization

Learn a powerful technique to effectively compress large data



Similarity search is a problem where given a query the goal is to find the most similar documents to it among all the database documents.

Introduction

In data science, similarity search often appears in the NLP domain, search engines or recommender systems where the most relevant documents or items need to be retrieved for a query. There exists a large variety of different ways to improve search performance in massive volumes of data.

In the [first part](#) of this article series, we looked at kNN and inverted file index structure for performing similarity search. As we learned, kNN is the most straightforward approach while inverted file index acts on top of it suggesting a trade-off between speed acceleration and accuracy. Nevertheless, both methods do not use data compression techniques which might lead to memory issues, especially in cases of large datasets and limited RAM. In this article, we will try to address this issue by looking at another method called **Product Quantization**.

Definition

Product quantization is the process where each dataset vector is converted into a short memory-efficient representation (called **PQ code**). Instead of fully keeping all the vectors, their short representations are stored. At the same time, product quantization is a lossy-compression method which results in lower prediction accuracy but in practice, this algorithm works very well.

In general, quantization is the process of mapping infinite values to discrete ones.

Training

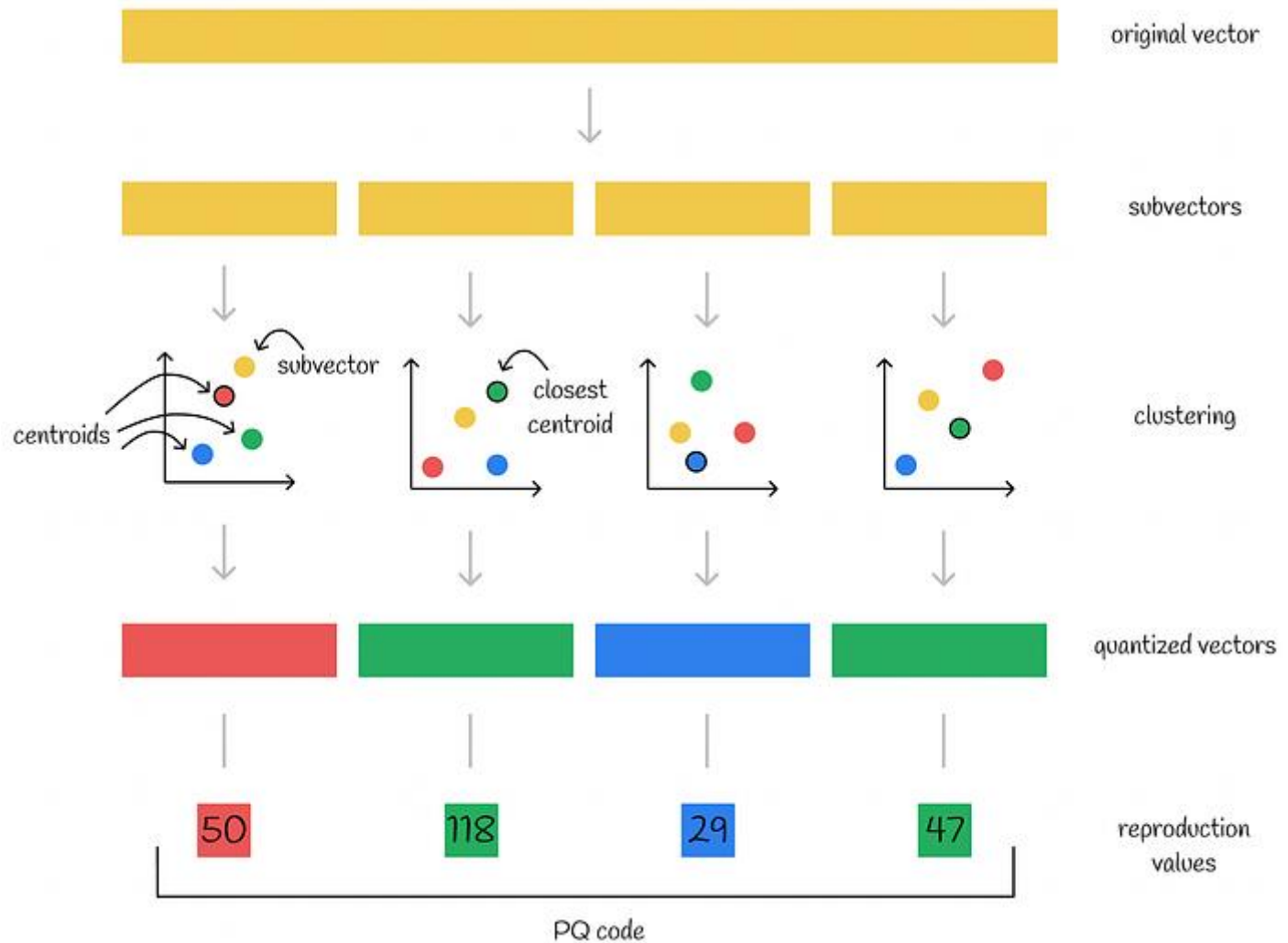
Firstly, the algorithm divides each vector into several equal parts — **subvectors**. Each of the respective parts of all dataset vectors form independent **subspaces** and is processed separately. Then a clustering algorithm is executed for each subspace of vectors. By doing so, several centroids in each subspace are created. Each subvector is encoded with the ID of the centroid that it belongs to. Additionally, the coordinates of all centroids are stored for later use.

*Subspace centroids are also called **quantized vectors**.*

*In product quantization, a cluster ID is often referred to as a **reproduction value**.*

Note. In the figures below a rectangle represents a vector containing several values while a square indicates a single number.

Press enter or click to view image in full size



Encoding using quantization

As a result, if an original vector is divided into n parts, then it can be encoded by n numbers — IDs of respective centroids for each of its subvectors. Typically, the number of created centroids k is usually chosen as a power of 2 for more efficient memory usage. This way, the memory required to store an encoded vector is $n * \log(k)$ bits.

*The collection of all centroids inside a subspace is called a **codebook**. Running n clustering algorithms for all subspaces produces n separate codebooks.*

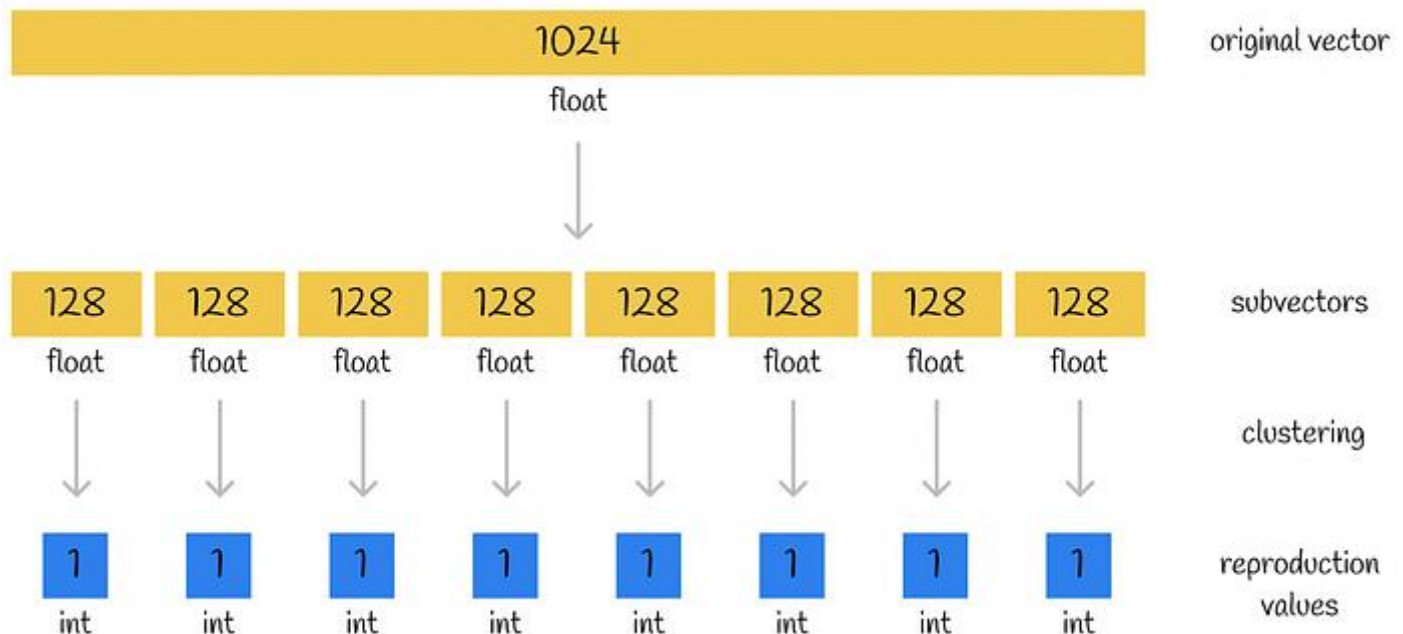
Compression example

Imagine an original vector of size 1024 which stores floats (32 bits) was divided into $n = 8$ subvectors where each subvector is encoded by one of $k = 256$ clusters. Therefore, encoding the ID of a single cluster would require $\log(256) = 8$ bits. Let us compare the memory sizes for the vector representation in both cases:

- Original vector: $1024 * 32 \text{ bits} = 4096 \text{ bytes}$.
- Encoded vector: $8 * 8 \text{ bits} = 8 \text{ bytes}$.

The final compression is 512 times! This is the real power of product quantization.

Press enter or click to view image in full size



Quantization example. Numbers in vectors show how many numbers it stores.

Here are some important notes:

- The algorithm can be trained on one subset of vectors (e.g., to create clusters) and be used for another one: once the algorithm is trained, another dataset of vectors is passed where new vectors are encoded by using already constructed centroids for each subspace.
- Typically, k-means is chosen as a clustering algorithm. One of its advantages is that the number of clusters k is a hyperparameter that can be manually defined, according to memory usage requirements.

Inference

To get a better understanding, let us first have a look at several naive approaches and find out their downsides. This will also help us realize why they should not be normally used.

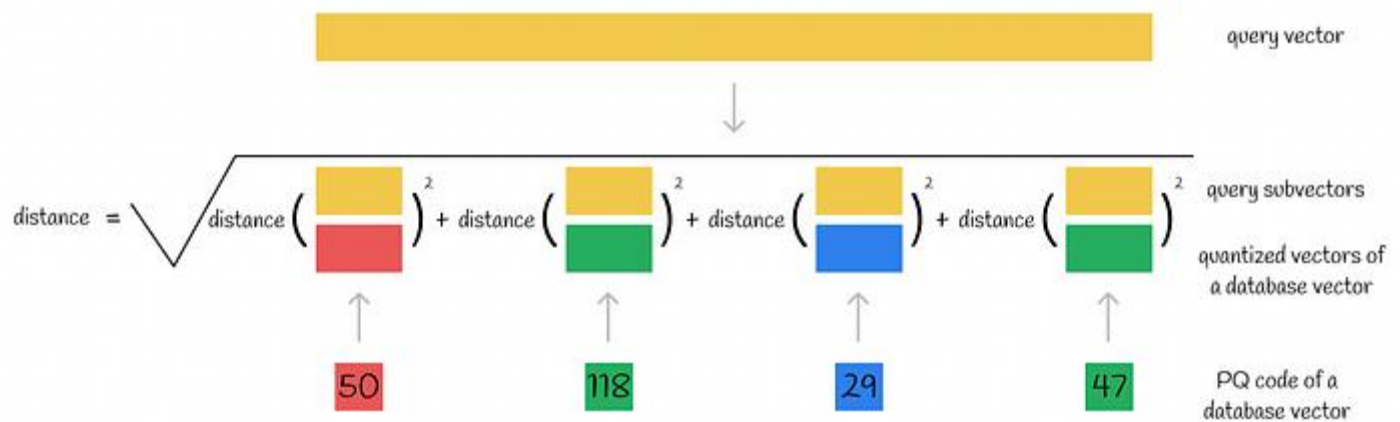
Naive approaches

The first naive approach consists of decompressing all vectors by concatenating the corresponding centroids of each vector. After that, the L_2 distance (or another metric) can be calculated from a query vector to all the dataset vectors. Obviously, this method works but it is very time-consuming because the brute-force search is performed and the distance calculation is performed on high-dimensional decompressed vectors.

Another possible way is to split a query vector into subvectors and compute a sum of distances from each query subvector to respective

quantized vectors of a database vector, based on its PQ code. As a consequence, the brute-search technique is used again and the distance calculation here still requires a linear time of the original vectors' dimensionality, as in the previous case.

Press enter or click to view image in full size



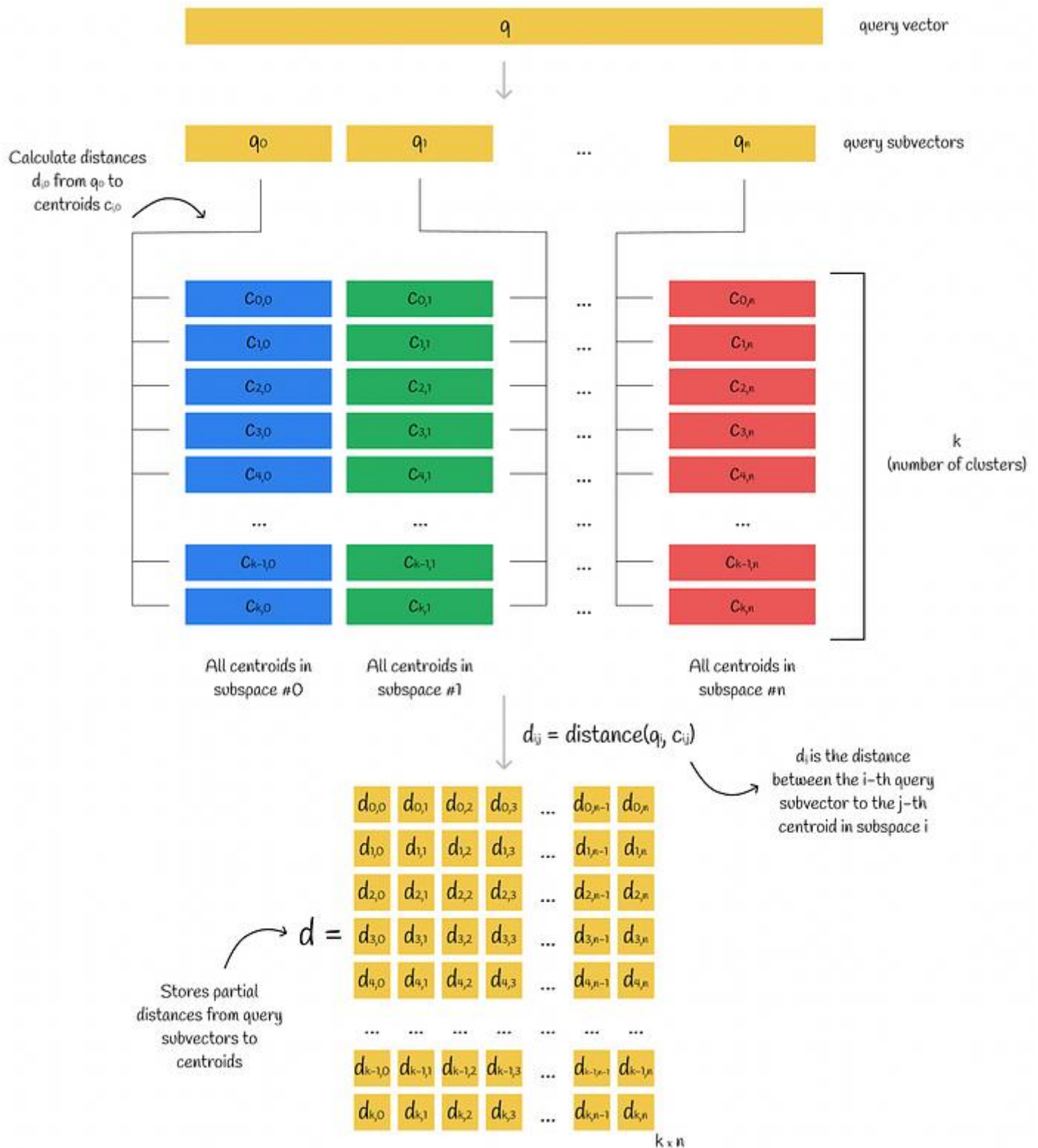
Calculating approximate distance using naive approach. The example is shown for euclidean distance as a metric.

Another possible method is to encode the query vector into a PQ code. Then this PQ code is directly utilized to calculate distances to all other PQ codes. The dataset vector with the corresponding PQ code which has the shortest distance is then considered as the nearest neighbour to the query. This approach is faster than the previous two because the distance is always computed between low-dimensional PQ codes. However, PQ codes are composed by cluster IDs which do not have a lot of semantic meaning and can be considered as a categorical variable explicitly used as a real variable. Clearly, this is a bad practice and this method can lead to poor prediction quality.

Optimized approach

A query vector is divided into subvectors. For each of its subvectors, distances to all the centroids of the corresponding subspace are computed. Ultimately, this information is stored in table d .

[Press enter or click to view image in full size](#)



Obtaining a table d storing partial query subvector-to-centroid distances

*Calculated subvector-to-centroid distances are often referred to as **partial distances**.*

By using this subvector-to-centroid distance table d , the approximate distance from the query to any database vector can be easily obtained by its PQ codes:

1. For each of subvectors of a database vector, the closest centroid j is found (by using mapping values from PQ codes) and the partial distance $d[i][j]$ from that centroid to the query subvector i (by using the calculated matrix d) is taken.
2. All the partial distances are squared and summed up. By taking the square root of this value, the approximate euclidean distance is obtained. If you want to know how to get approximate results for other metrics as well, navigate to the section below “*Approximation of other distance metrics*”.

Press enter or click to view image in full size

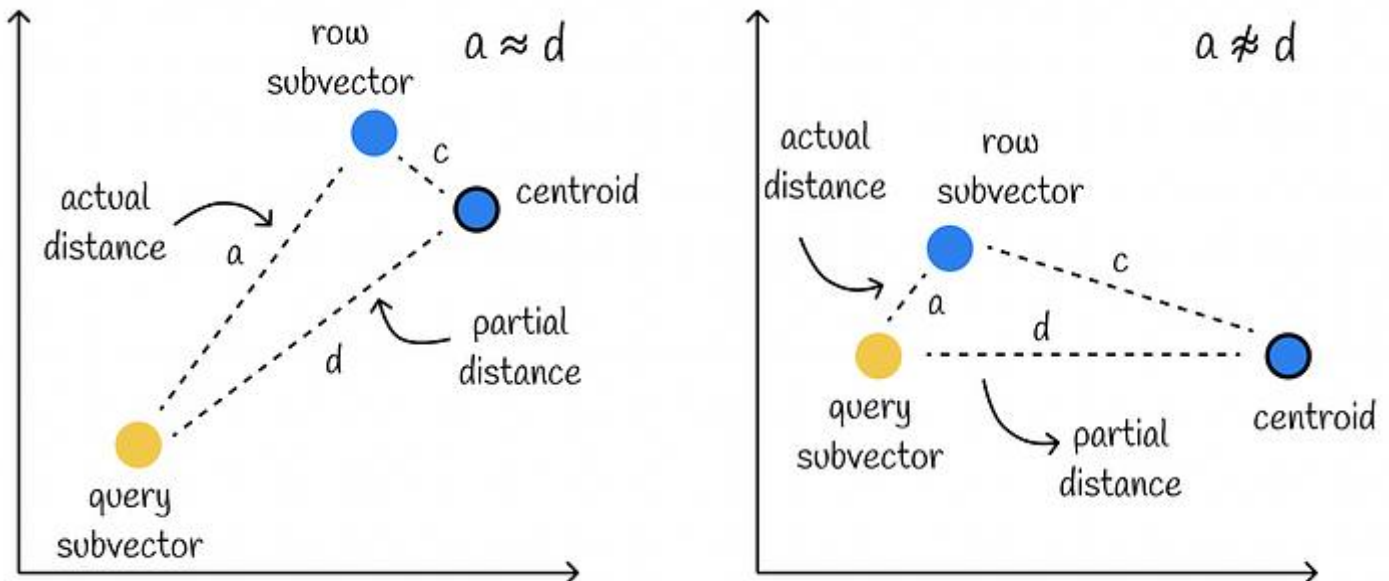


Computing distance from a query to database vector by using PQ code and distance table

Using this method for calculating approximate distances assumes that partial distances $\mathbf{d}$ are very close to actual distances $\mathbf{a}$ between query and database subvectors.

Nevertheless, this condition may not be satisfied, especially when the distance c between the database subvector and its centroid is large. In such cases, calculations result in lower accuracy.

Press enter or click to view image in full size



Example on the left shows a good case of approximation when the actual distance is very close to the partial distance (c is small). On the right side, we can observe a bad scenario because the partial distance is much longer than the actual distance (c is large).

After we have obtained approximate distances for all database rows, we search for vectors with the smallest values. Those vectors will be the nearest neighbours to the query.

Approximation of other distance metrics

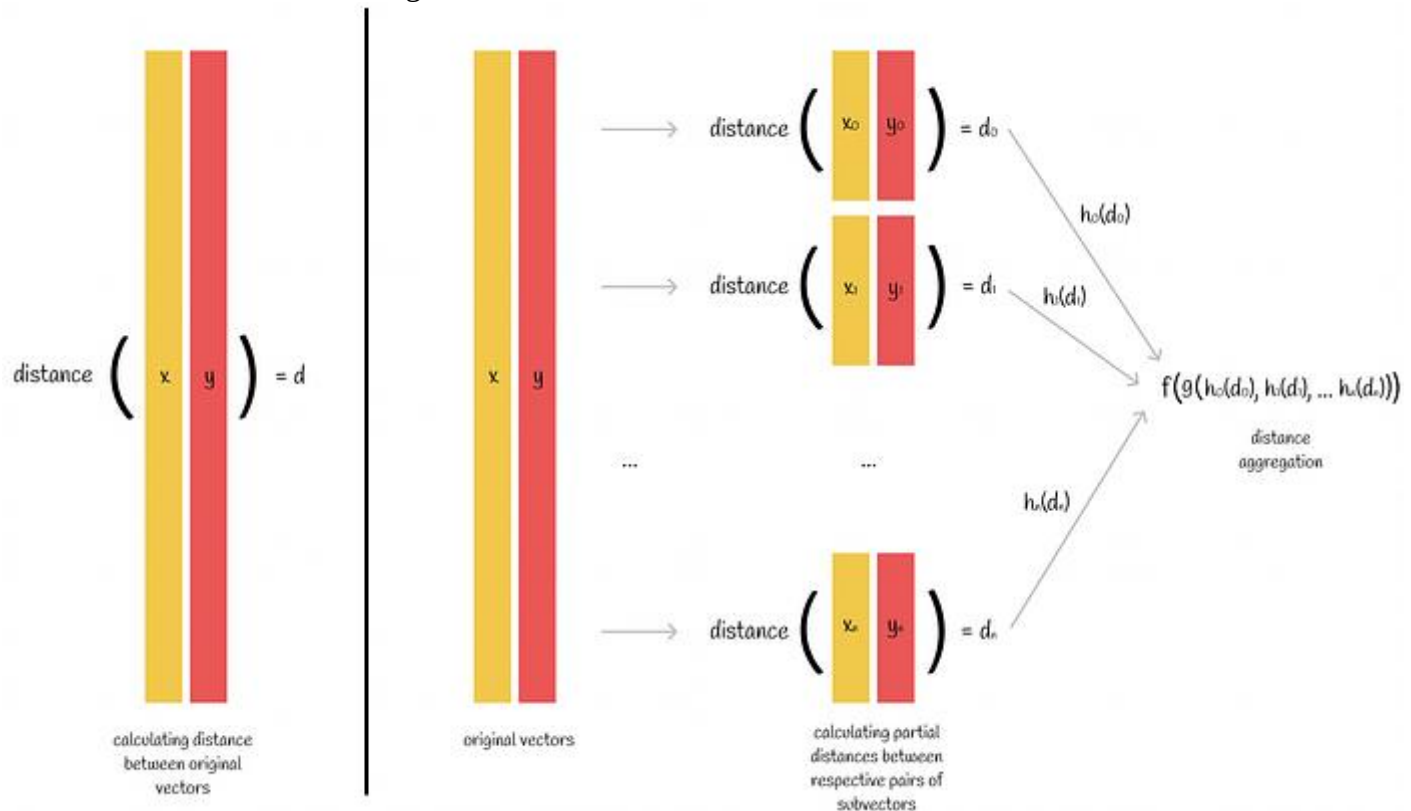
So far have looked at how to approximate euclidean distance by using partial distances. Let us generalize the rule for other metrics as well.

Imagine we would like to calculate a distance metric between a pair of vectors. If we know the metrics' formula, we can directly apply it to get

the result. But sometimes we can do it by parts in the following manner:

- Both vectors are divided into n subvectors.
- For each pair of respective subvectors, the distance metric is calculated.
- Calculated n metrics are then combined to produce the actual distance between the original vectors.

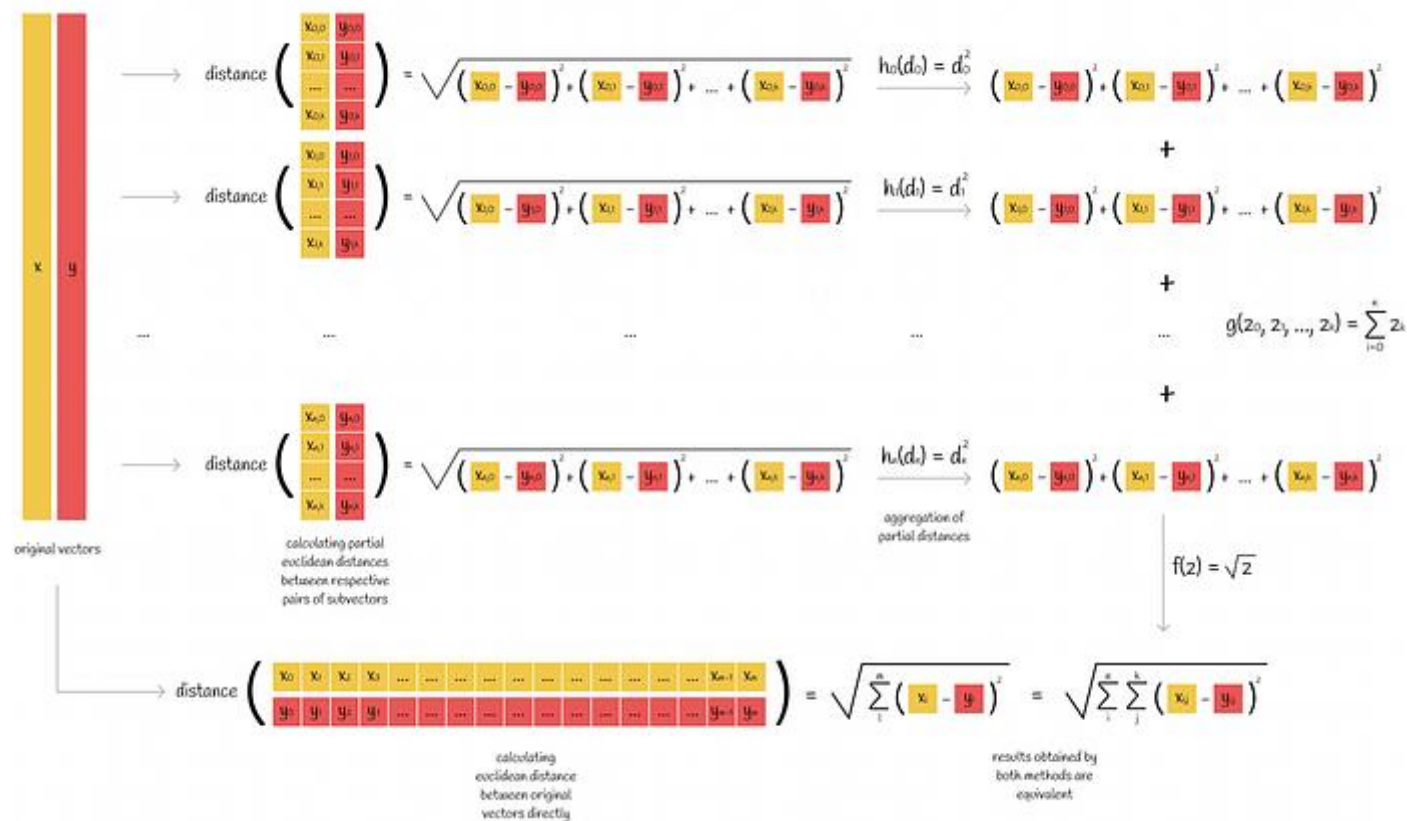
Press enter or click to view image in full size



The figure shows two ways of calculating a metric. On the left, the metric formula is directly applied to both vectors. On the right, partial distances are calculated for each pair of respective subvectors. Then they are combined by using aggregation functions h , g and f .

Euclidean distance is an example of a metric which can be calculated by parts. Based on the figure above, we can choose the aggregation functions to be $h(z) = z^2$, $g(z_0, z_1, \dots, z_n) = \text{sum}(z_0, z_1, \dots, z_n)$ and $f(z) = \sqrt{z}$.

Press enter or click to view image in full size



Euclidean distance can be calculated by parts

Inner product is another example of such metric with aggregation functions $h(z) = z$, $g(z_0, z_1, \dots, z_n) = \text{sum}(z_0, z_1, \dots, z_n)$ and $f(z) = z$.

In the context of product quantization, this is a very important property because during inference the algorithm calculates distances by parts. This means that it would be much more problematic to use

metrics for product quantization that do not have this property. Cosine distance is an example of such metric.

If there is still a need to use a metric without this property, then additional heuristics need to be applied to aggregate partial distances with the least possible error.

Performance

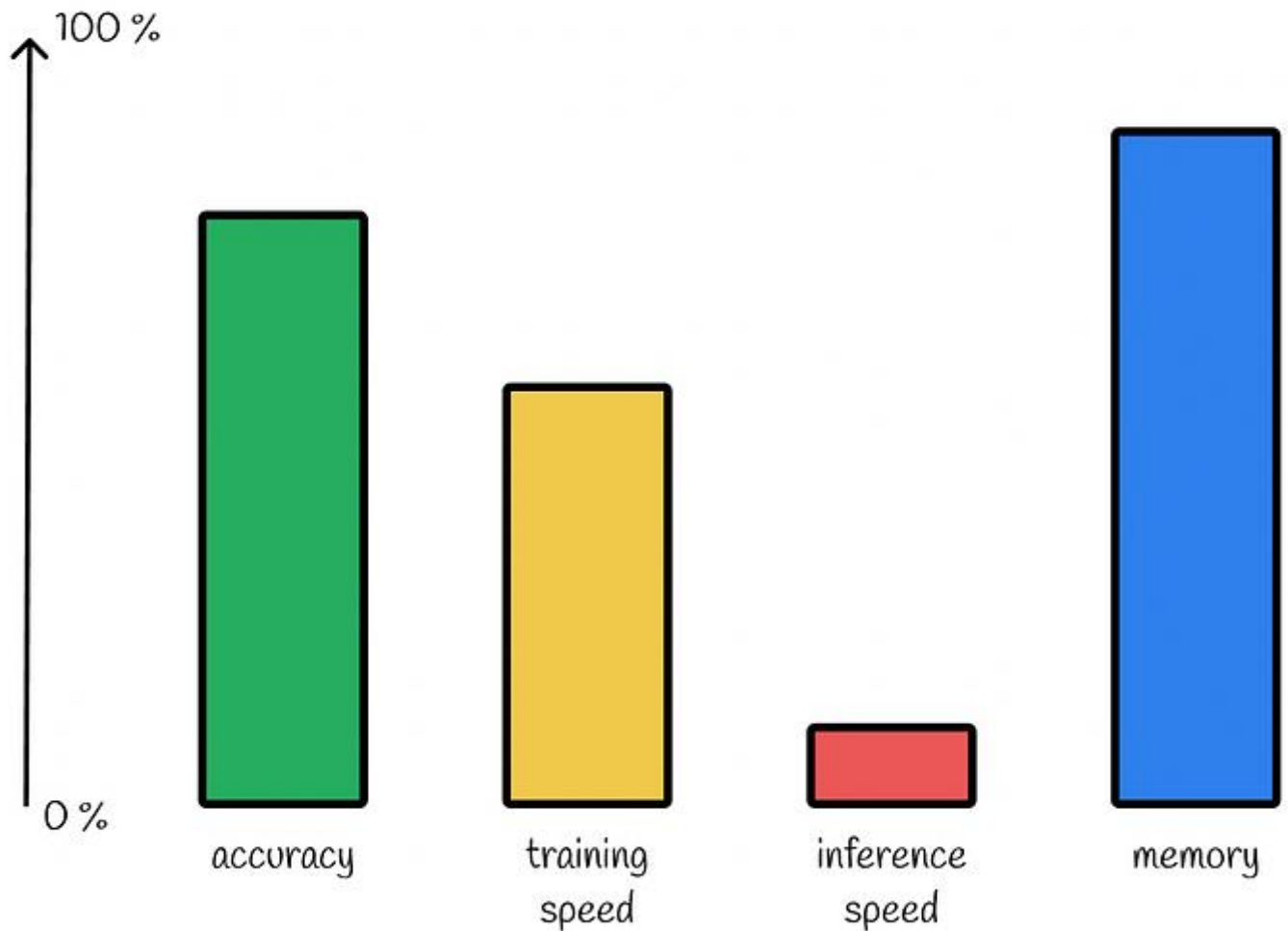
The main advantage of the product quantization is a massive compression of database vectors which are stored as short PQ codes. For some applications, such compression rate may be even higher than 95%! However, apart from PQ codes, the matrix d of size $k \times n$ containing quantized vectors of each subspace needs to be stored.

Product quantization is a lossy-compression method, so the higher the compression is, the more likely that the prediction accuracy will decrease.

Building a system for efficient representation requires training several cluster algorithms. Apart from it, during inference, $k * n$ partial distances need to be calculated in a brute-force manner and summed up for each of the database vectors which may take some time.

Press enter or click to view image in full size

Product Quantization



Product Quantization performance

Faiss implementation

Faiss (*Facebook AI Search Similarity*) is a Python library written in C++ used for optimised similarity search. This library presents different types of indexes which are data structures used to efficiently store the data and perform queries.

Based on the information from the [Faiss documentation](#), we will see how product quantization is utilized.

Product quantization is implemented in the *IndexPQ* class. For initialisation, we need to provide it 3 parameters:

- **d**: number of dimensions in data.
- **M**: number of splits for each vector (the same parameter as *n* used above).
- **nbits**: number of bits it takes to encode a single cluster ID. This means that the number of total clusters in a single subspace will be equal to $k = 2^{nbits}$.

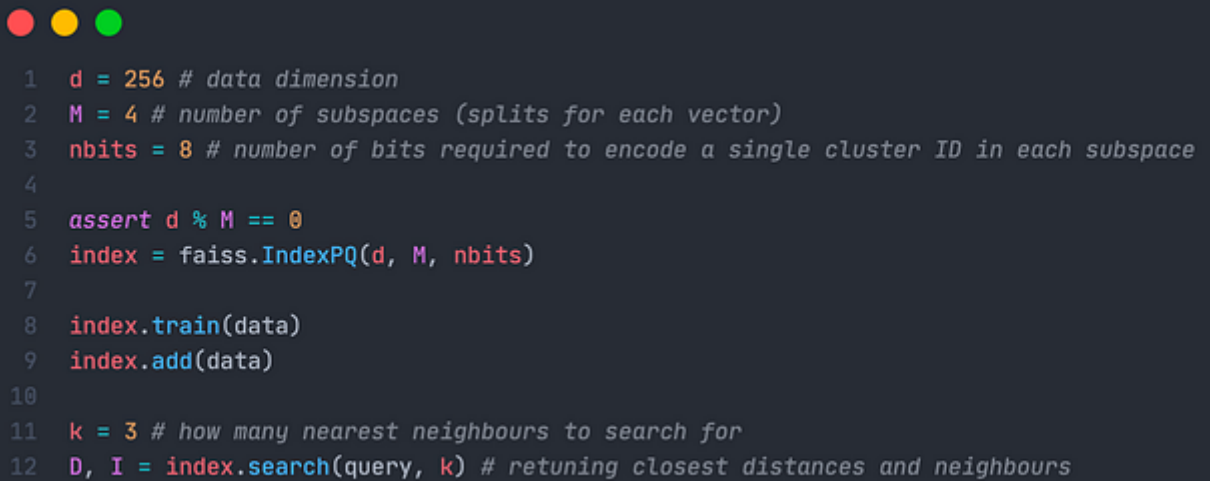
For equal subspace dimensions splitting, the parameter *dim* must be divisible by *M*.

The total number of bytes required to store a single vector is equal to:
Press enter or click to view image in full size

$$\text{bytes} = \left\lceil \frac{M * \text{nbits}}{8} \right\rceil$$

*As we can see in the formula above, for more efficient memory usage the value of $M * \text{nbits}$ should be divisible by 8.*

Press enter or click to view image in full size

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains Python code for creating and using a Faiss IndexPQ. The code is as follows:

```
1 d = 256 # data dimension
2 M = 4 # number of subspaces (splits for each vector)
3 nbits = 8 # number of bits required to encode a single cluster ID in each subspace
4
5 assert d % M == 0
6 index = faiss.IndexPQ(d, M, nbits)
7
8 index.train(data)
9 index.add(data)
10
11 k = 3 # how many nearest neighbours to search for
12 D, I = index.search(query, k) # returning closest distances and neighbours
```

Faiss implementation of IndexPQ

Conclusion

We have looked through a very popular algorithm in information retrieval systems that efficiently compresses large volumes of data. Its principal downside is a slow inference speed. Despite this fact, the algorithm is widely used in modern Big data applications, especially in combination with other similarity search techniques.

In the first part of the article series, we described the workflow of the inverted file index. In fact, we can merge these two algorithms into a more efficient one which will possess the advantages of both! This is what exactly we are going to do in the next part of this series.